Title

Exploring K-Means Clustering Using Customer Segmentation Dataset

Introduction

Clustering is a fundamental technique in unsupervised machine learning used to group similar data points into clusters without predefined labels. Among clustering algorithms, K-Means is one of the most widely applied due to its simplicity and efficiency. It partitions data into k clusters by minimizing the distance between each point and its assigned cluster centroid.

In this lab, we use the dataset PRL_12_8_2025, which contains customer information such as Annual Income and Spending Score. The goal is to segment customers into distinct groups to better understand their spending behavior, which can later help in targeted marketing, business decisions, and recommendation systems.

Required Library

- Pandas→ To read, clean, and manage tabular data.
- Numpy→ For efficient numerical computations.
- Matplotlib→ To visualize data and clustering results.
- Scikit-learn→ Provides KMeans algorithm and other machine learning tools.

Theory

K-means Clustering:

K-Means clustering groups data into k non-overlapping clusters by minimizing the within-cluster distances. It starts with randomly chosen centroids, assigns each point to the nearest centroid, then recalculates centroids until convergence.

Why it is used :

- Helps in discovering natural groupings within data.
- Widely used in customer segmentation, fraud detection, image compression, and pattern recognition.

Advantages :

- Simple and fast to implement.
- Works well when clusters are spherical and evenly distributed.
- Easy to interpret visually.

Disadvantages:

- Requires pre-selecting k.
- Sensitive to outliers and noise.

Elbow method:

The Elbow Method determines the best value of k. It works by plotting Within-Cluster Sum of Squares (WCSS) for different k values. The "elbow point" in the curve shows where adding more clusters doesn't improve performance significantly.

- If k is too small → underfitting, different groups merge together.
- If k is too large → overfitting, noise and outliers get classified as clusters.

Equation:

$$WCSS = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2 \tag{1}$$

How it works:

1. Run K-Means for k = 1 to 10.
2. Calculate WCSS for each k.
3. Plot WCSS vs k.
4. Look for the "elbow point" where WCSS reduction slows down.
   - For example, if WCSS drops sharply up to k = 3 and then flattens, k = 3 is optimal.

Code

Code	Explanation

`import pandas as pd`
`import numpy as np`
`import matplotlib.pyplot as plt`
`from sklearn.cluster import KMeans`
 	
`df = pd.read_csv("PRL_12_8_2025.csv")`
`print(df.head())`
	In this step, we import all the required libraries and load the dataset PRL_12_8_2025 into a Pandas DataFrame. The head() function shows the first five rows, giving us an idea about the structure of the dataset.

`plt.figure(figsize=(10,6), dpi=150)`
`plt.scatter(df['Annual␣Income␣(k$)'], df['Spending␣Score␣(1-100)'],`
`alpha=0.7, edgecolor='k')`
`plt.xlabel('Annual␣Income␣(k$)')`
`plt.ylabel('Spending␣Score␣(1-100)')`
`plt.title('Unlabeled␣Mall␣Customer␣Data')`
`plt.show()`
	Before applying clustering, it is useful to visualize raw data. Here, we plot customers' Annual Income on the x-axis and their Spending Score on the y-axis. This scatter plot helps us see natural groupings, though no clusters are assigned yet.

Code	Explanation
`X = df.iloc[:, [3, 4]].values` `print("First␣5␣feature␣rows:\n", X[:5])`	From the dataset, we select only two numerical features: Annual Income and Spending Score, since these are the most relevant for customer segmentation. The result X is a 2D array that will be passed into the clustering algorithm.

`clustering_score = []`
`for i in range(1, 11):`
`  kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)`
`  kmeans.fit(X)`
`  clustering_score.append(kmeans.inertia_)`
 	
`plt.figure(figsize=(10,6), dpi=150)`
`plt.plot(range(1, 11), clustering_score, marker='o', linestyle='--')`
`plt.scatter(5, clustering_score[4], s=200, c='red', marker='*')`
`plt.title('The␣Elbow␣Method')`
`plt.xlabel('Number␣of␣Clusters')`
`plt.ylabel('Clustering␣Score')`
`plt.show()`
	In this step, we run KMeans with clusters from 1 to 10 and record the Within-Cluster Sum of Squares (WCSS) for each. WCSS measures how compact the clusters are, with lower values indicating tighter groupings. Plotting WCSS against k gives the Elbow Curve. The sharp bend, or "elbow," shows where adding more clusters offers little improvement. Here, the elbow occurs at k = 5, suggesting five clusters provide the best balance of accuracy and simplicity.

`kmeans = KMeans(n_clusters=5, random_state=42)`
`pred = kmeans.fit_predict(X)`
`print(pred[:20])`
	Now we apply KMeans with 5 clusters. Each data point is assigned to a cluster labeled from 0 to 4. The pred array stores these labels. Printing the first 20 predictions helps confirm the clustering is working.

`df['Cluster'] = pred` `print(df['Cluster'].value_counts())`	To make the dataset more meaningful, we add a new column Cluster which stores the assigned cluster for each customer. This allows us to count how many customers belong to each group, making it easier to analyze their distribution.

Code	Explanation

`plt.figure(figsize=(10,6), dpi=150)`
`colors = ['brown', 'green', 'blue', 'purple', 'orange']`
`for i in range(5):`
`  plt.scatter(X[pred == i, 0], X[pred == i, 1],`
`  c=colors[i], label=f'Cluster {i}', alpha=0.7,`
`  edgecolor='k')`
 	
`plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],`
`  s=300, c='red', label='Centroid', marker='*')`
`plt.xlabel('Annual Income (k$)')`
`plt.ylabel('Spending Score (1-100)')`
`plt.legend()`
`plt.title('Customer Clusters')`
`plt.show()`
	Finally, we visualize the 5 clusters with different colors for each group. Red stars indicate the cluster centroids. The plot shows clear separation between clusters. This separation highlights different spending behaviors among customers. For example, some clusters represent high-income low spenders, while others are low-income high spenders.

Output

```
First 5 rows of dataset:
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
```

Figure 1: Dataset Display

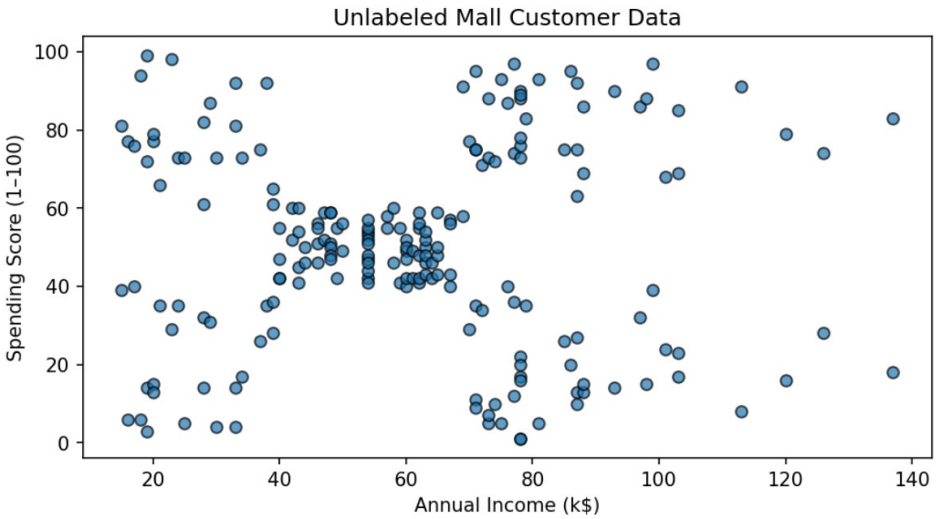

Figure 2: Scatter Plot - Annual Income vs Spending Score

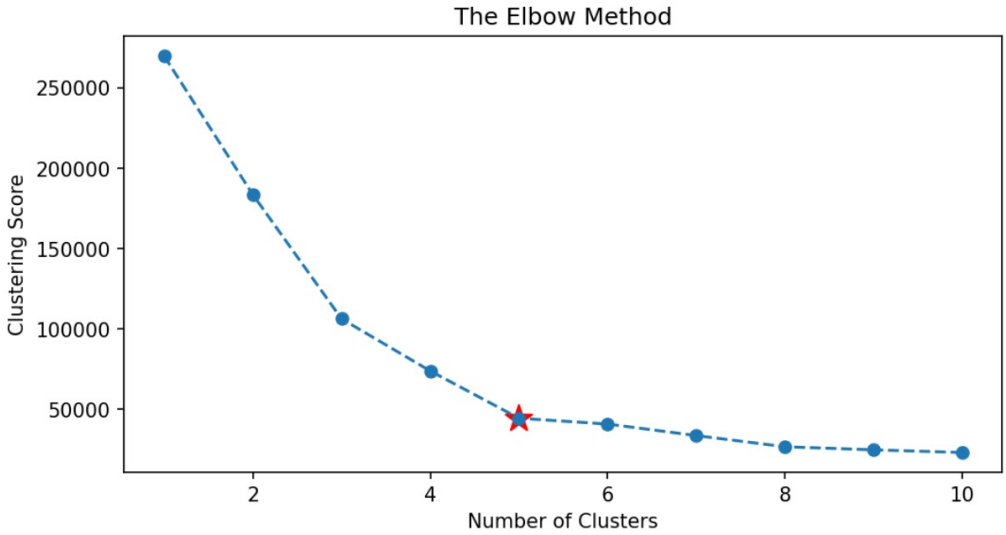

Figure 3: Elbow Method

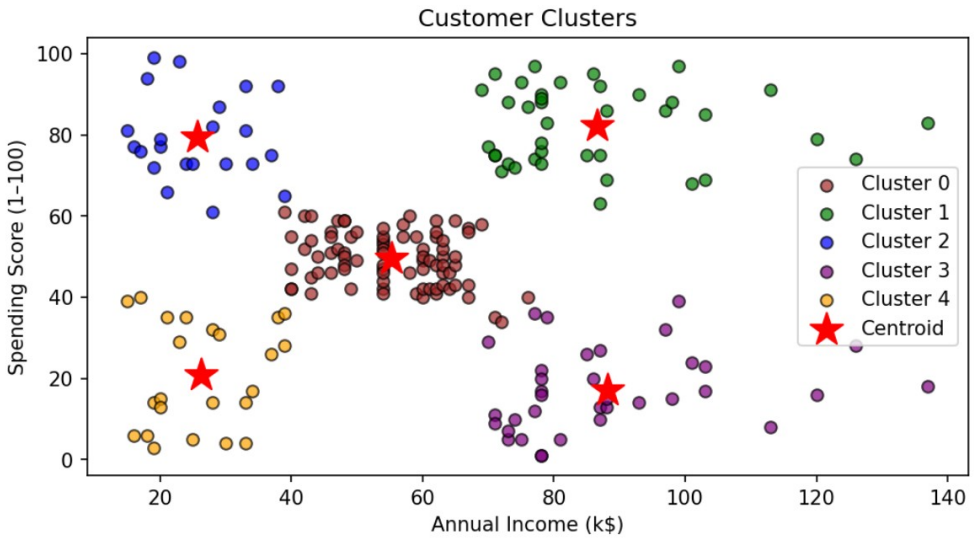

Figure 4: Clusters of Customers

Discussion

In this lab, we used K-Means clustering on the dataset PRL_12_8_2025, focusing on Annual Income and Spending Score to group customers. Using the Elbow Method, we determined that k = 5 is the most suitable number of clusters. After training the model, each customer was assigned to one of the five groups, and the visualization confirmed that the clusters were well-separated with clear centroids. These results highlight how clustering can reveal hidden customer patterns such as high-income low-spenders or moderate-income high-spenders. Overall, the experiment demonstrates that K-Means is an effective method for customer segmentation and decision-making support in business applications.